Split Flap Display Firmware

Firmware for a 12-module Chainlink/Denver3D-style split-flap display (ESP32), including the self-hosted WiFi web UI (message, presets, automation, timer).

What you need

- **PlatformIO** — install the PlatformIO IDE extension for VS Code (free).
- **USB cable** for your ESP32 (the T-Display board used here).
- This folder, unzipped, opened in VS Code.

Before you start (first-time setup)

A few things that trip people up the first time they flash an ESP32:

- **Use a data cable, not a charge-only cable.** If your board never shows up as a serial/COM port, this is the #1 cause — try a different USB cable before troubleshooting anything else.
- **Install the USB-to-serial driver.** These boards use a WCH USB-to-serial chip (CH9102 or CH340 depending on revision). Windows almost always needs a driver installed before the board will show up as a COM port; macOS is hit-or-miss depending on chip revision and OS version; Linux usually works out of the box. Get the driver directly from the chip maker:
  - Windows: https://www.wch.cn/downloads/CH341SER_EXE.html
  - macOS: https://www.wch.cn/downloads/CH34XSER_MAC_ZIP.html
  - After installing, unplug and replug the board.
- **The first build downloads the ESP32 toolchain.** PlatformIO's first Upload on a fresh checkout pulls down the ESP32 platform/toolchain (a few hundred MB) — needs an internet connection and can take several minutes before it even starts flashing. This is normal, not a hang.
- **Selecting the port.** PlatformIO usually auto-detects it. If not, or if you have multiple serial devices plugged in, pick it manually:
  - macOS: `/dev/cu.wchusbserial*` or `/dev/cu.usbserial*`
  - Windows: `COM3`, `COM4`, etc. (check Device Manager if unsure)
  - Linux: `/dev/ttyUSB0`, etc.
- **Close anything else using the port before uploading.** The web UI, a serial monitor, or any other program connected to the board will block the upload with a port-busy error — disconnect it first.

Setup

1. Open this folder in VS Code (PlatformIO will auto-detect `platformio.ini`).
2. In `firmware/esp32/splitflap/`, copy `secrets.h.example` to a new file named `secrets.h`, and edit it:
   - `WIFI_SSID` / `WIFI_PASSWORD` — your WiFi network
   - `DEVICE_INSTANCE_NAME` — the mDNS name for the display (e.g. `splitflap` gives you `splitflap.local`)
   - Leave `MQTT_*` and `OTA_PASSWORD` as-is unless you plan to use MQTT or wireless (OTA) updates — see comments in the file for details.
3. `secrets.h` is required for the project to build — it's excluded from this package on purpose since it holds credentials.

Build & upload

1. Connect the ESP32 via USB.
2. In VS Code's PlatformIO sidebar, select the **chainlink** environment (already the default — builds for 12 modules with the web UI enabled).
3. Click **Upload** (or from a terminal: `pio run -e chainlink --target upload`).

4. Open the Serial Monitor to confirm it boots. Once connected to WiFi, the display's screen shows its IP address — open that address (or `http://<DEVICE_INSTANCE_NAME>.local/`) in a browser for the web UI.

More info

- `firmware/esp32/splitflap/WEB_UI_README.md` — details on the wireless web UI (message/presets/automation/timer), known limitations, and how it stores data.
- `firmware/esp32/README.md` — how the ESP32 build layers on top of the shared splitflap driver code.
- `platformio.ini` — all build environments. `chainlink` is what you want for a standard build; `chainlink_ota` is the same build but uploads wirelessly once you've flashed once over USB. The `advanced_*` environments are for different hardware (Chainlink base station, driver tester) — skip those unless you know you need them.

License